

Applicazione alla complessità aritmetica

Definition 1. Un **algoritmo di moltiplicazione veloce** è un algoritmo che usa asintoticamente un numero di moltiplicazioni rispetto alle addizioni.

Definition (..) => ..:

Definition 2 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , la moltiplicazione tra due matrici $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle : \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p}$$
$$(A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo “algoritmo”, ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Definition (..) => ..: Moltiplicazione matriciale

Domanda:

Possiamo descrivere un algoritmo di moltiplicazione con un tensore e sperare di ridurre la complessità aritmetica?

(risposta sì)

Rappresentazione low-rank di forme bilineari

Data una decomposizione

$$T = \sum_{r=1}^R a_r \otimes b_r \otimes c_r$$

Possiamo scrivere una forma bilineare $\varphi : U \times V \rightarrow W$ come

$$\varphi(u, v) = \sum_{r=1}^R f_r(u) g_r(v) c_r,$$

dove $f_r(u) = a_r^\top u$ e $g_r(v) = b_r^\top v$ sono dei funzionali.

Bilinear computation

Forse prima?

Definition 1 (Bilinear computation). Qui definizione

Definition (..) => ..: Bilinear computation

Rappresentazione low-rank del prodotto matriciale

Date due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times p}$, poste $u = \text{vec}(A)$, $v = \text{vec}(B)$ abbiamo un algoritmo di moltiplicazione veloce a partire da una bilinear computation

$$\text{vec}(C^\top) = \varphi(u, v) = \sum_{r=1}^R (f_r(u) \cdot g_r(v)) c_r$$
$$= \sum_{r=1}^R (a_r^\top u \cdot b_r^\top v) c_r,$$

dove $\cdot$ denota le active multiplications.

Il rango

So far:

$$\begin{aligned} R(\varphi) &= \text{“Lunghezza di una bilinear computation”} \\ &= \text{“Rango del tensore associato”}. \end{aligned}$$

Vorremmo mostrare che

$$R(\varphi) = \text{“Numero di moltiplicazioni matriciali”}.$$

Spoiler: equazioni $\rightarrow$ bilinear comp $\rightarrow$ tensore

L'esponente

Definition 1. L'esponente della moltiplicazione matriciale è il numero

$$\omega := \inf_n \{ \tau \in \mathbb{R} \mid \text{La moltiplicazione tra due matrici in } \mathbb{R}^{n \times n} \text{ ha costo } o(n^\tau) \}.$$

Definition (. .) => . . :

Stime note su ω



L'algoritmo classico

$$\begin{aligned} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \\ &= \begin{bmatrix} f_1g_1 + f_2g_2 & f_3g_3 + f_4g_4 \\ f_5g_5 + f_6g_6 & f_7g_7 + f_8g_8 \end{bmatrix} \\ &= \begin{bmatrix} m_1 + m_2 & m_3 + m_4 \\ m_5 + m_6 & m_7 + m_8 \end{bmatrix} \\ &= \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \end{aligned}$$

Fissata la base standard $E_{11}, E_{12}, E_{21}, E_{22}$ dello spazio delle matrici 2×2 , possiamo scrivere la bilinear computation

$$\begin{aligned} C &= c_{11}E_{11} + c_{12}E_{12} + c_{21}E_{21} + c_{22}E_{22} \\ &= (m_1 + m_2)E_{11} + (m_3 + m_4)E_{12} \\ &\quad + (m_5 + m_6)E_{21} + (m_7 + m_8)E_{22} \\ &= \sum_{r=1}^8 m_r w_r. \end{aligned}$$

L'algoritmo corrisponde al tensore

$$\begin{aligned}
\langle 2, 2, 2 \rangle &= (a_{11} \otimes b_{11} + a_{12} \otimes b_{21}) \otimes c_{11} \\
&\quad + (a_{21} \otimes b_{11} + a_{22} \otimes b_{21}) \otimes c_{21} \\
&\quad + (a_{11} \otimes b_{12} + a_{12} \otimes b_{22}) \otimes c_{12} \\
&\quad + (a_{21} \otimes b_{12} + a_{22} \otimes b_{22}) \otimes c_{22}.
\end{aligned}$$

L'algoritmo di Strassen

Facendo alcune semplificazioni del risultato del prodotto matriciale, troviamo l'algoritmo di Strassen in forma tensoriale

QUI

Due domande:

Come calcolare la complessità quando si usano i due algoritmi ricorsivamente?

Come ridurre il rango migliora la complessità?

Stimare la complessità nel caso generale

QUI ALBERO?

Date due matrici di taglia $2^i \times 2^i$ possiamo fare i livelli di ricorsione e ottenere che

$$R(\langle 2^i, 2^i, 2^i \rangle) = R(\langle 2, 2, 2 \rangle^{\otimes i}) \leq R(\langle 2, 2, 2 \rangle)^i \leq r^i.$$

Nel caso generale, dato $n > 2$, possiamo fare un padding di zeri TODO: sistemare per farlo sembrare un embedding dentro matrice più grande

$$\begin{bmatrix} A & \\ & \end{bmatrix} \begin{bmatrix} B & \\ & \end{bmatrix}$$

fino a raggiungere la taglia $2^{\lceil \log_2 n \rceil}$.

La complessità sarà comunque limitata da R :

$$\begin{aligned} R(\langle n, n, n \rangle) &\leq R(\langle 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil} \rangle) \\ &\leq R(\langle 2, 2, 2 \rangle)^{\lceil \log_2 n \rceil} \\ &\leq R^{\lceil \log_2 n \rceil} \\ &\leq R \cdot n^{\log_2 R} \end{aligned}$$

Da cui

Proposition 1. Se $R(\langle n, n, n \rangle) \leq R$ per degli interi positivi n, R , allora $n^\omega \leq R$.

Proposition (. .) => . . :

Abbiamo dimostrato che $\omega \leq \log_n R(\langle n, n, n \rangle)$.

Da cui,

$$\omega \leq \inf_n \log_n R(\langle n, n, n \rangle).$$

Il teorema di Strassen

Vorremmo che

$$\omega = \liminf_{n \rightarrow +\infty} \log_n R(\langle n, n, n \rangle).$$

Algoritmi approssimati

bini et al provano...

tensore che trovano

Cenni sull'errore